

Capitulo 1: Arquitectura del Motor de Recuperacion Moderno	3
1.1 Arquitectura en Capas	3
1.2 Flujo Interno de Recuperacion	4
1.3 Parser NTFS: El Filesystem mas Recuperable	4
1.4 Parser APFS: El Filesystem mas Desafiante	5
1.5 Parser EXT4: El Desafio de los Inodes Eliminados	5
1.6 Parsers FAT y exFAT: Simplicidad con Limitaciones	6
1.7 File Carving: Recuperacion sin Metadatos	6
1.8 Lectura Cruda de Disco: La Base de Todo	7
Capitulo 2: Componentes Reutilizables — Que Existe y Que Licencias Permiten	7
2.1 The Sleuth Kit / libtsk: El Mejor Candidato	7
2.2 PhotoRec: 480+ Formatos de Carving	8
2.3 GNU ddrescue: El Estandar de Imagen de Discos Fallando	8
2.4 Analisis de Licencias: Que Se Puede Usar Comercialmente	8
2.5 Arquitectura Modular Recomendada	9
Capitulo 3: Componentes a Desarrollar — El Verdadero Valor	10
3.1 Motor de Decisiones: El Corazon del Producto	10
3.2 Imagen Priorizada: MFT Primero	11
3.3 Modulo de Diagnostico Automatizado	11
3.4 Checkpoints Incrementales: Escaneos Reanudables	11
Capitulo 4: Como Aprende el Sistema — El Motor de Decisiones	12
4.1 Que Datos Guarda el Sistema	12
4.2 Que Variables Aprende	12
4.3 Como Se Entrena	13
4.4 Como Evita Aprender Cosas Incorrectas	13
Capitulo 5: Dataset — El Activo Mas Dificil de Conseguir	13
5.1 Estado Actual: No Existen Datasets Publicos	14
5.2 Generacion Sintetica de Datos de Prueba	14
5.3 Estrategia de Alianzas para Recoleccion de Datos Reales	14

5.4 Privacidad y Consideraciones Legales	15
5.5 Tamano del Dataset y Roadmap	15
Capitulo 6: MVP Tecnico — Demostrar que Funciona	16
6.1 Alcance del MVP	16
6.2 Parser NTFS: La Cadena Minima	16
6.3 Imagen de Disco: Algoritmo de 5 Fases	17
6.4 Modulo de Diagnostico: 20 Segundos Antes de Tocar el Disco	17
6.5 Stack Tecnico Recomendado	17
6.6 Benchmarking: Como Demostrar Superioridad	18
6.7 Cronograma de Desarrollo	18
Capitulo 7: Plan de Ingenieria — De la Vision alCodigo	19
7.1 Mapa de 14 Modulos	19
7.2 Fases de Desarrollo	20
7.3 Algoritmos Clave: El Corazon del Motor	20
7.4 Foso Competitivo: Porque No Pueden Copiarnos	21
7.5 Evaluacion de Riesgos	21
7.6 Conclusion: La Pregunta Central	22

Capitulo 1: Arquitectura del Motor de Recuperacion Moderno

Construir un motor de recuperacion de datos no es escribir un solo programa que "lee archivos rotos". Es disenar un sistema modular donde cada capa cumple una funcion especifica, desde la lectura cruda de sectores hasta la reconstruccion del arbol de directorios. Este capitulo describe la arquitectura interna de los motores de recuperacion modernos, analizando como herramientas como R-Studio, DMDE y UFS Explorer organizan sus componentes internos, y proponiendo una arquitectura optima para un motor de nueva generacion que incorpore un motor de decisiones adaptativo.

1.1 Arquitectura en Capas

Los motores de recuperacion profesionales comparten una arquitectura comun en capas. En la base se encuentra la capa de lectura cruda (Raw I/O), que se comunica directamente con el disco a traves de comandos ATA/SCSI. Encima opera la capa de deteccion de particiones y ensamblaje RAID, que reconstruye la estructura logica del disco. Luego vienen los parsers de filesystem (NTFS, APFS, EXT4, FAT/exFAT), que interpretan las estructuras de metadatos para localizar archivos. Si los metadatos estan danados, el motor de carving entra en accion, buscando archivos por firmas binarias. Finalmente, el motor de reconstruccion intenta rearmar archivos fragmentados o estructuras de filesystem parcialmente destruidas.

Capa	Funcion	Componentes Clave
Raw I/O	Lectura cruda de sectores, manejo de errores	ATA/SCSI commands, O_DIRECT, SG_IO, reintentos adaptativos
Deteccion de Particiones	Localizar tablas de particiones, reconstruir LBA	MBR, GPT, APM, deteccion heuristica
Ensamblaje RAID	Reconstruir arrays virtuales	Stripe detection, paridad, LVM, Storage Spaces
Parsers de Filesystem	Interpretar estructuras de metadatos	NTFS, APFS, EXT4, FAT32, exFAT, HFS+
Motor de Carving	Recuperar archivos por firmas binarias	Header/footer, structural parsing, smart carving
Motor de Reconstruccion	Rearmar archivos fragmentados, FS danados	Bifragment gap, heuristica de fragmentos, journal replay
Motor de Decisiones (Diagnostico)	Diagnosticar, seleccionar estrategia, adaptar	Bayesian network, decision trees, RL policy
Capa de Presentacion	Mostrar resultados, interaccion con usuario	Arbol de archivos, hex viewer, progreso

Tabla 1.1: Capas de la arquitectura del motor de recuperacion

La clave de esta arquitectura es que cada capa es independiente y comunicable. El parser NTFS no necesita saber si los datos provienen de un disco fisico, una imagen ddrescue, o un array RAID reconstruido. El motor de carving no necesita saber si el filesystem esta intacto o destruido. Esta separacion permite que cada componente sea desarrollado, testeado y mejorado de forma independiente, y que el motor de decisiones pueda invocar exactamente las capas que necesita para cada escenario.

1.2 Flujo Interno de Recuperacion

El flujo interno de un motor de recuperacion moderno sigue un pipeline bien definido. Cuando el usuario selecciona un disco o imagen, el sistema comienza por la capa de lectura cruda, que obtiene acceso exclusivo al dispositivo. Luego se ejecuta la deteccion de particiones, que identifica las regiones del disco que contienen filesystems. Para cada particion detectada, se invoca el parser correspondiente, que intenta leer las estructuras de metadatos. Si el parser tiene exito, se construye un arbol virtual de archivos. Si el parser falla parcialmente, se activa el motor de reconstruccion, que intenta recuperar estructuras danadas usando heurísticas. Si el parser falla completamente, se activa el motor de carving como ultimo recurso.

Lo que diferencia a un motor de recuperacion superior de uno mediocre no es la lista de formatos soportados, sino la inteligencia con la que navega este pipeline. Un motor mediocre siempre ejecuta todos los pasos en el mismo orden: escanea todo el disco, parsea el filesystem, y luego ejecuta carving. Un motor inteligente primero diagnostica: evalua el estado del disco, identifica el tipo de dano, y selecciona la estrategia optima. Si el disco esta fallando, prioriza la imagen antes que la recuperacion. Si el MFT esta parcialmente intacto, intenta reconstruccion antes que carving. Si el filesystem es APFS con TRIM activo, desvía recursos hacia carving porque la recuperacion basada en metadatos probablemente fracasara.

1.3 Parser NTFS: El Filesystem mas Recuperable

NTFS es el filesystem mas favorable para la recuperacion de datos, y esto se debe a una característica fundamental: cuando un archivo se elimina en NTFS, las entradas del MFT (Master File Table) no se zeroan inmediatamente. La entrada del MFT mantiene su atributo \$DATA con las data runs (listas de extensiones) intactas, lo que permite reconstruir la ubicacion fisica del archivo en el disco. Además, NTFS mantiene un diario de transacciones (\$LogFile) que permite reconstruir el estado del filesystem en un punto anterior en el tiempo, y un espejo del MFT (\$MFTMirr) que contiene una copia de las primeras 4 entradas del MFT.

Estructura	Proposito	Relevancia para Recuperacion
VBR (Sector 0)	Punto de entrada, contiene BPB con geometria	Indica ubicacion del \$MFT y \$MFTMirr
\$MFT	Tabla maestra de archivos, 1024 bytes por entrada	Principal fuente de metadatos de archivos
\$MFTMirr	Copia de las primeras 4 entradas del MFT	Backup si el inicio del MFT esta danado
\$LogFile	Diario de transacciones	Reconstruccion de estado anterior
\$Bitmap	Mapa de clusters en uso	Verificar si clusters de archivos eliminados fueron reutilizados
INDX (\$I30)	Indices de directorios	Alternativa cuando MFT esta parcialmente danado
Data Runs	Lista de extensiones del archivo	Mapeo VCN→LCN para archivos no residentes

Tabla 1.2: Estructuras NTFS clave para recuperacion

El parser NTFS debe seguir una cadena de lectura especifica: primero leer el VBR para obtener la ubicacion del MFT, luego leer la entrada 0 del MFT (que describe al propio \$MFT) para determinar el tamano total de la tabla, y luego iterar sobre todas las entradas del MFT. Cada entrada contiene atributos como

\$STANDARD_INFORMATION (timestamps, flags), \$FILE_NAME (nombre, directorio padre), y \$DATA (contenido o data runs). Para archivos eliminados, el bit de "en uso" en el offset 0x16 esta desactivado, pero el resto de la entrada permanece intacto hasta que el cluster es reutilizado.

Los escenarios de corrupcion de NTFS que un parser robusto debe manejar incluyen: VBR danado (usar backup en el ultimo sector), MFT fragmentado (usar \$MFTMirr y busqueda heuristica), entradas MFT con fixup values incorrectos (validar y reparar), y data runs con offsets relativos que apuntan a clusters fuera del rango del volumen. Un parser de calidad profesional debe poder recuperar archivos incluso cuando el 30-40% de las entradas del MFT estan danadas, utilizando sources alternativos como INDX y el \$LogFile para reconstruir la informacion faltante.

1.4 Parser APFS: El Filesystem mas Desafiante

APFS (Apple File System) representa el desafio mas grande para la recuperacion de datos. A diferencia de NTFS, APFS utiliza Copy-on-Write (CoW), lo que significa que cada modificacion escribe datos en un nuevo bloque fisico en lugar de sobrescribir el existente. Esto en principio pareceria beneficioso para recuperacion, pero en la practica APFS combina CoW con TRIM agresivo: cuando un archivo se elimina, APFS no solo marca los bloques como libres, sino que emite comandos TRIM al SSD, lo que provoca que el controlador del SSD borre fisicamente los bloques en la siguiente coleccion de basura. El resultado es que la recuperacion de archivos eliminados en APFS con SSD es casi imposible sin herramientas especializadas de nivel fisico.

La estructura de APFS es considerablemente mas compleja que la de HFS+. El contenedor APFS se organiza en un NX Superblock que apunta a un Object Map (OMAP), que mapea Object IDs virtuales a bloques fisicos. Las referencias a extensiones de archivos estan en un B-tree separado del arbol de archivos/carpetas, lo que requiere una busqueda de dos pasos para localizar los datos de un archivo: primero encontrar el nodo del B-tree de archivos para obtener el extent reference, y luego buscar en el B-tree de extensiones para obtener los bloques fisicos. Cada nodo del B-tree tiene un btrailer con checksum, lo que permite verificar la integridad de cada nodo individualmente.

Caracteristica	Impacto en Recuperacion
Copy-on-Write	Historial de versiones accesible hasta TRIM/purga
TRIM agresivo	Eliminacion fisica de bloques en SSDs — casi imposible recuperar
OMAP (Object Map)	Capa de indireccion: virtual OID → bloque fisico
B-tree de extensiones separado	Dos pasos para localizar datos de un archivo
Snapshots integrados	Punto de acceso para estado anterior del filesystem
Encriptacion nativa	Sin clave de desbloqueo, los datos son inaccesibles
btrailer checksums	Verificacion de integridad por nodo

Tabla 1.3: Caracteristicas de APFS y su impacto en recuperacion

1.5 Parser EXT4: El Desafio de los Inodes Eliminados

EXT4 presenta un desafío particularmente difícil para la recuperación de archivos eliminados: cuando un archivo se elimina en EXT4, los punteros del extent tree se zeroan. El extent tree, que almacena el mapeo de bloques lógicos a bloques físicos, es limpiado por el kernel de Linux como parte de la operación de eliminación. Esto significa que, a diferencia de NTFS donde las data runs permanecen intactas, en EXT4 la información de ubicación física del archivo se pierde inmediatamente. La principal estrategia de recuperación en EXT4 depende del journal (diario de transacciones), que puede contener copias de los inodes antes de la eliminación.

EXT4 organiza el disco en block groups, cada uno con su propio superblock, descriptores de grupo, tabla de inodes, bitmaps de bloques e inodes, y bloques de datos. Los extent trees reemplazan a los indirect block pointers de EXT2/3, proporcionando un mapeo más eficiente para archivos grandes. Sin embargo, cuando un archivo se elimina, el kernel zeroa los punteros del extent tree en el inode, marca los bloques como libres en el bitmap, y remueve la entrada del directorio. El journal puede contener la copia previa del inode con los punteros intactos, pero solo si la eliminación fue journalizada y el journal no ha sido reciclado. Esta es la razón por la que las herramientas de recuperación de EXT4 siempre intentan leer el journal primero.

1.6 Parsers FAT y exFAT: Simplicidad con Limitaciones

FAT32 es el filesystem más simple de recuperar: la tabla FAT (File Allocation Table) es un array que mapea cada cluster al siguiente cluster en la cadena del archivo. Cuando un archivo se elimina, el primer byte de la entrada del directorio se reemplaza por 0xE5 (marcador de eliminación), y las entradas en la tabla FAT se zeroan, rompiendo la cadena de clusters. Sin embargo, los datos reales del archivo permanecen en los clusters hasta que son sobrescritos. Para archivos contiguos (que no están fragmentados), la recuperación es trivial: se lee el cluster de inicio y se leen clusters consecutivos hasta alcanzar el tamaño del archivo. Para archivos fragmentados, la recuperación requiere heurística o carving.

exFAT introduce diferencias significativas: utiliza un bitmap de asignación en lugar de una tabla FAT, y los archivos contiguos no necesitan una cadena FAT (se indican con un flag en la entrada stream extension). Los archivos eliminados en exFAT se marcan invirtiendo el tipo de entrada (bit 0 del campo EntryType). exFAT tiene una sola tabla FAT (no duplicada como FAT32), lo que hace la validación más difícil. La recuperación en exFAT es similar a FAT32 para archivos contiguos, pero más compleja para fragmentados porque no hay una tabla de cadena.

1.7 File Carving: Recuperación sin Metadatos

El file carving es la técnica de recuperación de último recurso: cuando los metadatos del filesystem están completamente destruidos, el carving busca archivos directamente por sus firmas binarias (magic numbers) en el flujo crudo de sectores. Los métodos de carving progresan desde el simple header-footer carving (buscar el inicio y fin de un archivo por sus firmas) hasta el smart carving, que utiliza validación estructural para verificar que los datos encontrados realmente corresponden a un archivo válido del tipo detectado.

El mayor desafío del carving es la fragmentación: si un archivo está fragmentado en dos o más partes no contiguas, el carving simple fallará porque asumirá que los datos son contiguos. Las técnicas avanzadas como bifragment gap carving intentan reensamblar archivos fragmentados en dos partes buscando el gap óptimo entre los fragmentos, pero el problema es NP-hard para más de dos fragmentos. PhotoRec, la herramienta de

carving mas popular, soporta mas de 480 tipos de archivo y utiliza un enfoque de carving estructural que valida la integridad interna de cada archivo encontrado, pero no resuelve la fragmentacion. Un motor de recuperacion superior deberia combinar carving con heuristica de fragmentacion basada en el tipo de filesystem y los patrones de asignacion observados.

1.8 Lectura Cruda de Disco: La Base de Todo

La capa de lectura cruda es el fundamento sobre el cual se construye todo el motor de recuperacion. En Linux, el acceso directo a disco se realiza abriendo el dispositivo `/dev/sdX` con `O_DIRECT` para evitar el cache del kernel, y leyendo sectores con `read()`. Para enviar comandos ATA directos (passthrough), se utiliza la interfaz `SG_IO` del kernel, que permite enviar comandos como `READ DMA EXT (0x25)` para LBA48. En Windows, el acceso se realiza mediante `CreateFile()` sobre `\\.\PhysicalDriveN` y `ReadFile()`, con `IOCTL_ATA_PASS_THROUGH` para comandos directos.

La diferencia critica entre SATA y USB es que los puentes USB traducen comandos ATA a SCSI, perdiendo acceso a funcionalidades avanzadas como el bypass de cache, el control de timeout por comando, y la lectura de SMART. Los discos conectados por USB son mas dificiles de diagnosticar y recuperar porque el kernel no puede enviar comandos ATA nativos. Los adaptadores USB-SATA tambien pueden truncar datos o introducir errores de traduccion. Un motor de recuperacion profesional debe detectar el tipo de conexion y ajustar su estrategia en consecuencia: para USB, usar lecturas de alto nivel con timeouts generosos; para SATA, usar ATA passthrough para control maximo.

Capitulo 2: Componentes Reutilizables — Que Existe y Que Licencias Permiten

No es necesario construir todo desde cero. El ecosistema de codigo abierto ofrece componentes de alta calidad para la recuperacion de datos, pero la licencia de cada componente determina si puede ser integrado en un producto comercial. Este capitulo analiza los principales componentes disponibles, su calidad, sus licencias, y las opciones de integracion para cada caso. La conclusion principal es que existe un nucleo solido de componentes con licencias comerciales (libtsk, Scalpel, Foremost, libyal), pero las herramientas mas avanzadas (ddrescue, PhotoRec, OpenSuperClone) estan bajo GPL, lo que requiere reimplementacion o aislamiento de procesos.

2.1 The Sleuth Kit / libtsk: El Mejor Candidato

The Sleuth Kit (TSK) es una coleccion de herramientas forenses de codigo abierto mantenida por Brian Carrier, y libtsk es su biblioteca C subyacente. Es el mejor candidato para integracion en un motor de recuperacion comercial por tres razones: tiene una API de biblioteca documentada (no es solo una herramienta CLI), soporta los filesystems mas importantes (NTFS, FAT, exFAT, EXT2/3/4, HFS+, UFS, YAFFS2), y su licencia IBM CPL permite enlace comercial sin obligacion de liberar el codigo fuente del producto. La API de libtsk permite operaciones como `tsk_fs_open_img()` para abrir una imagen de disco, `tsk_fs_dir_walk()` para recorrer directorios, y `tsk_fs_file_walk()` para iterar sobre los bloques de un archivo.

Sin embargo, libtsk tiene limitaciones importantes para recuperacion: fue disenado como herramienta forensica, lo que significa que es de solo lectura y no intenta reconstruir estructuras danadas. Si el filesystem esta corrupto, libtsk fallara silenciosamente en lugar de intentar heuristica de reconstruccion. Ademas, no soporta APFS nativamente, lo que es una brecha significativa dado el mercado de macOS. Y su soporte para archivos eliminados en EXT4 es limitado porque no implementa journal replay. A pesar de estas limitaciones, libtsk proporciona una base solida para los parsers de filesystem que si soporta, y puede ser complementado con modulos propios para los casos que no cubre.

2.2 PhotoRec: 480+ Formatos de Carving

PhotoRec es la herramienta de carving mas completa del ecosistema open-source, con soporte para mas de 480 tipos de archivo. No se limita a header/footer carving: implementa carving estructural que valida la integridad interna de cada archivo encontrado, lo que reduce significativamente los falsos positivos. Su base de datos de firmas es el activo mas valioso, pero esta atrapada en codigo GPL sin modo biblioteca. PhotoRec no puede ser enlazado directamente en un producto comercial; solo puede ser ejecutado como subprocesso.

La estrategia recomendada es implementar un motor de carving propio utilizando la base de datos de firmas de PhotoRec como referencia (las firmas son datos, no codigo), y Scalpel (Apache 2.0) como base de codigo para la implementacion. Scalpel ya utiliza un formato de configuracion de firmas compatible con el de PhotoRec, lo que facilita la migracion. Alternativamente, Foremost (Public Domain) proporciona un motor de carving simple pero sin restricciones de licencia, adecuado como punto de partida para un MVP.

2.3 GNU ddrescue: El Estandar de Imagen de Discos Fallando

GNU ddrescue es la herramienta de referencia para crear imagenes de discos con sectores danados. Su algoritmo de 5 fases (Copying, Trimming, Sweeping, Scraping, Retrying) es elegante y efectivo: primero lee los sectores faciles, luego refina los limites de las zonas danadas, y finalmente reintenta los sectores que fallaron. El mapfile registra el estado de cada sector (leido, fallido, en proceso), lo que permite reanudar la imagen despues de una interrupcion sin perder progreso. Sin embargo, ddrescue esta bajo GPL v2+, lo que impide su enlace directo en un producto comercial.

La buena noticia es que el algoritmo de ddrescue esta bien documentado y es apto para reimplementacion en limpio (clean-room). El algoritmo no es trivial pero tampoco es excesivamente complejo: un desarrollador competente puede reimplementar las 5 fases en 4-6 semanas. La reimplementacion ademas permite mejoras: agregar priorizacion por region (MFT primero en NTFS), integrar analisis de timing por sector, e implementar aprendizaje entre sesiones (ddrescue no aprende de sesiones anteriores). OpenSuperClone va un paso mas alla que ddrescue al enviar comandos ATA/SCSI directos y implementar head-skipping basado en clustering de errores LBA, pero su complejidad es considerablemente mayor.

2.4 Analisis de Licencias: Que Se Puede Usar Comercialmente

La licencia es el factor decisivo para la integracion de componentes open-source. La siguiente tabla resume las licencias de los principales componentes y sus implicaciones para un producto comercial. Las licencias permisivas (MIT, BSD, Apache, CPL, LGPL) permiten enlace directo. Las licencias GPL requieren

aislamiento de procesos o reimplementacion.

Componente	Licencia	Enlace Comercial	Estrategia
libtsk (Sleuth Kit)	IBM CPL	Si — enlace directo	Usar como biblioteca principal de FS
Scalpel	Apache 2.0	Si — enlace directo	Motor de carving base
Foremost	Public Domain	Si — sin restricciones	Carving simple para MVP
libyal	LGPL v3+	Si — enlace dinamico	Parsers de formatos especificos
RecoverJPEG	BSD	Si — enlace directo	Carving de JPEG
PhotoRec	GPL v2+	No — subproceso o reimplementacion	Reimplementar referencia de firmas
TestDisk	GPL v2+	No — subproceso o reimplementacion	Reimplementar referencia de particiones
GNU ddrescue	GPL v2+	No — reimplementacion reconstruccion	Reimplementar algoritmo de 5 fases
OpenSuperClone	GPL v2	No — reimplementacion reconstruccion	Reimplementar head-skipping + ATA passthrough
Untrunc	GPL v2	No — reimplementacion factible	Reimplementar reparacion de MP4 (2-4 semanas)

Tabla 2.1: Licencias y estrategias de integracion por componente

2.5 Arquitectura Modular Recomendada

Basandose en el analisis de licencias y capacidades, la arquitectura recomendada combina componentes de enlace directo con reimplementaciones de algoritmos GPL. El nucleo del motor utiliza libtsk como biblioteca de filesystem, Scalpel como motor de carving, y componentes propios reimplementados para imagen de disco (algoritmo ddrescue), reparacion de MP4 (algoritmo untrunc), y acceso ATA directo (algoritmo OpenSuperClone). Los componentes GPL (PhotoRec, TestDisk) pueden ser ofrecidos como integraciones opcionales via subprocesos, pero el motor no debe depender de ellos para su funcionamiento basico.

Modulo	Base	Licencia	Desarrollo
Parser NTFS	libtsk + extension propia	CPL + propietario	3-4 semanas (extensiones)
Parser APFS	Propio (libtsk no soporta APFS)	Propietario	8-10 semanas
Parser EXT4	libtsk + journal replay propio	CPL + propietario	4-6 semanas
Parser FAT/exFAT	libtsk	CPL	Integracion directa
Motor de Carving	Scalpel + firmas propias	Apache 2.0 + propietario	4-6 semanas
Imagen de Disco	Reimplementacion de ddrescue	Propietario	6-8 semanas
ATA Passthrough	Reimplementacion de OSC	Propietario	8-10 semanas
Reparacion MP4	Reimplementacion de untrunc	Propietario	2-4 semanas
Motor de Decisiones	Propio (no existe equivalente)	Propietario	12-16 semanas

Capítulo 3: Componentes a Desarrollar — El Verdadero Valor

Los componentes que se deben desarrollar desde cero son los que constituyen el verdadero valor diferencial del motor. No se trata de los parsers de filesystem (que ya existen en libtsk) ni del carving (que Scalpel ya implementa), sino de las capas de inteligencia que ningún producto actual ofrece: el motor de decisiones, el diagnóstico automatizado, la imagen priorizada, y el sistema de aprendizaje. Estos componentes son los que transforman un recuperador de archivos en una plataforma de decisión para recuperación de datos.

3.1 Motor de Decisiones: El Corazón del Producto

El motor de decisiones es el componente que ningún competidor tiene. Su función es automatizar las decisiones que hoy toma un técnico experimentado: diagnóstico del estado del disco, selección de estrategia de recuperación, priorización de archivos, ajuste dinámico durante el proceso, y explicación al usuario de las decisiones tomadas. El motor no es un único algoritmo, sino un sistema de tres capas que opera en diferentes niveles de abstracción y velocidad de decisión.

Capa 1: Diagnóstico — Red Bayesiana

La primera capa utiliza una red bayesiana para el diagnóstico del estado del disco. Las redes bayesianas son ideales para este propósito porque manejan la incertidumbre de forma nativa (algo que un técnico experimentado maneja con intuición), permiten incorporar conocimiento experto como priors (ideal para el problema de arranque en frío), y son interpretables: el usuario puede ver exactamente por qué el sistema llegó a un diagnóstico determinado. Las variables de entrada incluyen: datos SMART (5 atributos clave), tiempo de lectura por sector, tasa de errores de I/O, tipo de filesystem, tipo de daño reportado por el usuario, y resultado del escaneo rápido inicial. Las variables de salida son probabilidades de: fallo físico inminente, corrupción lógica, eliminación accidental, daño por malware, y fallo de SSD con TRIM.

Capa 2: Selección de Estrategia — Árbol de Decisión

La segunda capa utiliza un árbol de decisión (o random forest) para seleccionar la estrategia de recuperación. Los árboles de decisión son interpretables (el usuario puede entender la lógica), rápidos de inferir (milisegundos), y fáciles de entrenar con datos etiquetados. Las variables de entrada incluyen el diagnóstico de la Capa 1, el tipo de filesystem, el tipo de daño, y los recursos disponibles (tiempo, disco de destino). Las decisiones de salida incluyen: priorizar imagen antes que recuperación, tipo de escaneo (rápido, profundo, carving), orden de lectura (forward, backward, MFT-first), y si se debe ofrecer al usuario la opción de reparación con IA para archivos multimedia.

Capa 3: Optimización de Lectura — Aprendizaje por Refuerzo

La tercera capa utiliza aprendizaje por refuerzo (RL) para optimizar la estrategia de lectura en tiempo real. Mientras el disco está siendo leído, el agente RL ajusta la estrategia de lectura basándose en la telemetría en

tiempo real: tasa de errores por zona, latencia de lectura por sector, y tasa de recuperacion de datos utiles. El framework de multi-armed bandit es directamente aplicable: los brazos son las estrategias de lectura (forward, backward, skip zone, retry with different parameters), el contexto es el estado de salud del disco, y la recompensa es bytes recuperados por unidad de tiempo. Esta es una aplicacion novedosa de RL que no existe en ningun producto comercial.

3.2 Imagen Priorizada: MFT Primero

Ningun motor de imagen de disco actual prioriza regiones del disco segun su importancia para la recuperacion. ddrescue y HDDSuperClone leen secuencialmente, intentando rescatar todos los datos por igual. Pero un tecnico experimentado sabe que si puede recuperar el MFT completo de un disco NTFS, puede reconstruir virtualmente todo el arbol de archivos, incluso si algunos sectores de datos estan danados. La imagen priorizada es una innovacion que modifica el orden de lectura para obtener primero las estructuras de metadatos criticas del filesystem, antes que los datos de usuario. Para NTFS, esto significa leer el MFT primero. Para EXT4, leer la tabla de inodes y el journal. Para APFS, leer el NX Superblock y los B-trees de catalogo y extensiones.

La implementacion requiere que el motor de imagen conozca la estructura del filesystem que esta intentando recuperar. Esto representa un cambio arquitectonico fundamental: el motor de imagen ya no es una herramienta generica que lee sectores, sino un componente que entiende la semantica del filesystem y puede tomar decisiones informadas sobre que sectores leer primero. El flujo seria: leer el VBR para identificar el filesystem y localizar las estructuras criticas, luego leer esas estructuras en orden de prioridad, y solo despues proceder con la imagen secuencial del resto del disco. Si el disco esta fallando y podria morir en cualquier momento, esta priorizacion puede significar la diferencia entre recuperar el arbol completo de archivos y recuperar solo datos fragmentados.

3.3 Modulo de Diagnostico Automatizado

El modulo de diagnostico es la primera capa que se ejecuta cuando el usuario conecta un disco. Su funcion es evaluar el estado del disco en 20-30 segundos, sin escribir nada, y presentar al usuario un informe claro de la situacion. El diagnostico incluye: lectura e interpretacion de datos SMART (5 atributos clave: Reallocated Sectors, Reported Uncorrectable Errors, Command Timeout, Current Pending Sector, Offline Uncorrectable), escaneo rapido de superficie (lectura de un sector cada N para detectar zonas danadas), verificacion de integridad del filesystem (superblock, MFT, journal), y evaluacion del riesgo de continuar operando sobre el disco. El resultado es un diagnostico con semaforo (verde, amarillo, rojo) que el usuario puede entender, acompañado de una recomendacion de accion.

3.4 Checkpoints Incrementales: Escaneos Reanudables

Los escaneos de discos grandes (18TB+) pueden tardar mas de 40 horas. Si el proceso se interrumpe por un corte de energia, un reinicio del sistema, o un fallo del disco, todo el progreso se pierde sin checkpoints. El sistema de checkpoints incrementales guarda el estado del escaneo de forma periodica (cada 30 segundos o cada 1000 sectores), permitiendo reanudar desde el ultimo punto guardado. El estado incluye: bitmap de sectores procesados (codificado con RLE para eficiencia), arbol de archivos descubierto hasta el momento,

estadísticas de salud del disco, y parámetros de la estrategia actual. R-Studio ya implementa algo similar con sus "scan info files", pero no de forma granular ni con la capacidad de reanudar exactamente desde el punto de interrupción.

La implementación recomendada utiliza un bitmap RLE para el mapa de sectores (cada sector tiene 3 estados: no procesado, procesado con éxito, procesado con error), almacenado en un archivo SQLite con escritura atómica. El checkpoint se guarda cada 30 segundos o cada 10,000 sectores, lo que ocurra primero. El archivo de checkpoint es pequeño (típicamente menos de 1MB incluso para discos de 18TB) porque el bitmap RLE comprime eficientemente las zonas contiguas. Al reanudar, el motor carga el checkpoint, restaura el bitmap de sectores y el árbol de archivos, y continúa desde el último sector procesado.

Capítulo 4: Como Aprende el Sistema — El Motor de Decisiones

La pregunta más crítica de la Fase 3 no es "qué algoritmo usar", sino "cómo aprende el sistema y qué datos necesita para aprender". Un motor de decisiones que no aprende de los resultados es solo un sistema de reglas; un motor que aprende de forma incorrecta es peor que un sistema de reglas. Este capítulo describe en detalle qué datos guarda el sistema, qué variables aprende, cómo se entrena, cómo evita aprender cosas incorrectas, y cómo mejora con el tiempo.

4.1 Que Datos Guarda el Sistema

El sistema debe almacenar dos tipos de datos: telemetría de cada caso de recuperación (datos de entrada) y resultados de la recuperación (datos de retroalimentación). La telemetría incluye información del disco (modelo, fabricante, capacidad, tipo HDD/SSD, interfaz SATA/USB/NVMe), datos SMART (5 atributos clave + todos los disponibles), tipo de filesystem y versión, tipo de daño reportado por el usuario, resultado del diagnóstico automático (probabilidades de cada tipo de fallo), estrategia seleccionada por el motor de decisiones, parámetros de ejecución (orden de lectura, timeout, reintentos), y telemetría en tiempo real (tasa de errores por zona, latencia de lectura, bytes recuperados).

Los resultados de la recuperación incluyen: archivos detectados vs archivos recuperados con éxito, integridad de los archivos recuperados (checksum SHA-256 cuando es posible), confirmación del usuario (los archivos recuperados son correctos y útiles), tiempo total de recuperación, y si el usuario abandonó el proceso antes de completarlo. Esta información es crítica porque permite al sistema aprender no solo qué estrategias funcionan, sino también qué estrategias hacen que el usuario abandone (por ejemplo, un escaneo que tarda 12 horas sin mostrar resultados intermedios).

4.2 Que Variables Aprende

El sistema aprende correlaciones entre variables de entrada y resultados de recuperación. Las variables que el sistema debe aprender a correlacionar incluyen: tipo de daño vs estrategia óptima (por ejemplo, MFT dañado en NTFS funciona mejor con reconstrucción + INDX, no con carving), fabricante de disco vs patrones de

fallo (Seagate vs WD vs Toshiba tienen diferentes modos de fallo típicos), tipo de filesystem vs probabilidad de recuperación (NTFS elimina conservando metadatos, EXT4 elimina zeroando punteros, APFS con TRIM elimina físicamente), y umbral de errores por zona vs probabilidad de éxito de reintentos (si una zona tiene más de N errores, es mejor saltarla que seguir reintentando).

4.3 Como Se Entrena

El entrenamiento del motor de decisiones tiene tres fases, correspondientes a las tres capas del motor. La Capa 1 (red bayesiana) se inicializa con priors expertos: los conocimientos de técnicos experimentados codificados como probabilidades a priori. Por ejemplo, un técnico sabe que si un disco hace click, la probabilidad de fallo físico inminente es mayor al 90%. Estos priors proporcionan un comportamiento razonable desde el primer caso. A medida que el sistema procesa casos reales, las probabilidades se actualizan mediante inferencia bayesiana, ajustándose a los datos observados.

La Capa 2 (árbol de decisión) se entrena con datos etiquetados de casos pasados. Cada caso completo (diagnóstico + estrategia + resultado) constituye un ejemplo de entrenamiento. El árbol aprende reglas como "si el disco es NTFS con MFT dañado y más del 50% de inodes intactos, entonces usar reconstrucción + INDX". El entrenamiento es batch: se reentrena periódicamente (por ejemplo, cada 100 casos nuevos) en lugar de continuamente, lo que evita que el sistema sea inestable. La Capa 3 (RL) se entrena online: cada sesión de recuperación es un episodio de entrenamiento donde el agente ajusta su política basándose en las recompensas obtenidas (bytes recuperados por unidad de tiempo, ajustado por el riesgo de daño al disco).

4.4 Como Evita Aprender Cosas Incorrectas

El mayor riesgo de un sistema de aprendizaje es aprender de datos sesgados o incorrectos. Si el sistema aprende de usuarios que confirman archivos corruptos como "recuperados con éxito", sus futuras decisiones serán incorrectas. Para mitigar este riesgo, se implementan múltiples mecanismos de seguridad. Primero, la retroalimentación del usuario se pondera por confiabilidad: un usuario que verifica archivos con checksums tiene peso mayor que uno que simplemente hace clic en "aceptar todo". Segundo, el sistema nunca aprende de un solo caso: se requiere un mínimo de 10 casos similares antes de ajustar una regla. Tercero, se implementan validadores internos que detectan anomalías en los datos de entrenamiento (por ejemplo, si la tasa de éxito de una estrategia cambia drásticamente de un día para otro, el sistema sospecha de datos corruptos y no actualiza).

Cuarto, el sistema mantiene un conjunto de validación de casos de referencia (ground truth) que nunca se usa para entrenamiento, solo para evaluar la calidad del modelo. Estos casos de referencia son creados con imágenes de disco sintéticas donde se conoce exactamente que archivos existen y cuáles están dañados. Si el rendimiento del modelo en el conjunto de validación disminuye, se revierte a la versión anterior del modelo. Quinto, las actualizaciones del modelo se versionan y se pueden revertir: cada versión del modelo se almacena con sus métricas de rendimiento, y si una nueva versión es peor que la anterior en el conjunto de validación, se descarta automáticamente.

Capítulo 5: Dataset — El Activo Más Difícil de Conseguir

El dataset es el activo mas critico y mas dificil de conseguir. Sin datos de entrenamiento, el motor de decisiones es solo teoria. Este capitulo analiza la situacion actual de los datasets de recuperacion de datos (spoiler: no existen publicamente), las estrategias para construir un dataset propio, y las consideraciones legales y eticas que deben cumplirse.

5.1 Estado Actual: No Existen Datasets Publicos

No existe ningun dataset publico de imagenes de disco danadas para investigacion de recuperacion. Los datasets forenses existentes (NIST CFReDS, Digital Corpora, Enron corpus, DFRWS challenges) contienen imagenes de filesystems intactos con archivos eliminados u ocultos, pero no imagenes de medios fisicamente danados o filesystems corruptos. El articulo de Grajeda et al. (2017), citado 191 veces, identifica explicitamente esta brecha: los datasets forenses se disenar para validacion de herramientas forenses, no para investigacion de recuperacion de datos. Digital Corpora almacena 169 imagenes de disco (1.106 TB), pero todas son filesystems intactos. El NIST CFReDS ofrece imagenes de referencia para validacion forense, no para recuperacion de dano. Esta brecha es una oportunidad: quien construya el primer dataset publico de recuperacion de datos se posicionara como referente en la investigacion.

5.2 Generacion Sintetica de Datos de Prueba

La estrategia mas viable para la fase inicial es generar imagenes de disco sinteticas con dano controlado. El proceso es: crear un disco virtual con un filesystem conocido y un conjunto de archivos de prueba (con checksums SHA-256 registrados), luego aplicar corrupcion controlada a estructuras especificas del filesystem, y finalmente intentar la recuperacion con el motor. Las herramientas disponibles para generar dano controlado incluyen: dd para escritura de bytes especificos en offsets exactos, qemu-img para crear imagenes de disco virtuales, dmsetup y dm-flakey para inyectar errores de I/O en dispositivos de bloque, y QEMU con NVMe virtual para simular TRIM en SSDs. Un pipeline automatizado puede generar cientos de variantes de dano a partir de una imagen base.

La ventaja de los datos sinteticos es que se conoce la verdad fundamental (ground truth): se sabe exactamente que archivos existian antes de la corrupcion, cuales estaban intactos, cuales fueron danados, y cuales fueron eliminados. Esto permite evaluar objetivamente el rendimiento del motor de recuperacion: si recupera un archivo, se puede verificar que el checksum coincide con el original. La desventaja es que los datos sinteticos no cubren todos los escenarios del mundo real: la corrupcion real es mas compleja y variada que la que se puede generar con scripts. Por eso, los datos sinteticos son necesarios pero no suficientes: deben ser complementados con datos reales anonimizados.

5.3 Estrategia de Alianzas para Recoleccion de Datos Reales

Los talleres de reparacion independientes son la fuente mas accesible de datos reales. Existen mas de 30,000 talleres de reparacion en Estados Unidos, y la comunidad Technibble (con mas de 200,000 miembros) es el punto de entrada principal. La propuesta de valor para los talleres es clara: a cambio de compartir metadatos anonimizados de sus casos de recuperacion (NO imagenes de disco completas), reciben acceso gratuito o con descuento al motor de recuperacion. Los metadatos que se recopilarian incluyen: modelo y fabricante del

disco, tipo de dano, datos SMART, estrategia utilizada, y resultado de la recuperacion. Nunca se comparten imagenes de disco completas, que contienen datos personales del cliente del taller.

Las universidades con programas de forensica digital (Champlain, Purdue, UCF) son aliados ideales para la recoleccion de datos bajo aprobacion IRB. Los investigadores universitarios necesitan datasets para publicar articulos, y un proyecto de recoleccion de datos de recuperacion es un topico de investigacion atractivo. Backblaze es el proveedor de datos en la nube mas abierto: ya publica estadisticas de fallos de discos de mas de 300,000 unidades. Su metodologia de recoleccion (snapshots diarios de SMART con Smartmontools + Drive Sentinel) puede adaptarse para recolectar metadatos de escenarios de recuperacion. Los fabricantes de hardware son los mas dificiles de abordar debido a preocupaciones competitivas, pero podrian estar interesados en un dataset anonimizado de patrones de fallo que les ayude a mejorar sus productos.

5.4 Privacidad y Consideraciones Legales

Las imagenes de disco son casi imposibles de anonimizar completamente: contienen informacion personal en todos los niveles (contenido de archivos, metadatos, estructuras del filesystem, datos eliminados, espacio slack). La recomendacion es nunca compartir imagenes de disco completas. En su lugar, se recopila solo metadatos: datos SMART, modelo de disco, tipo de dano, estrategia utilizada, y resultado. Estos metadatos no contienen informacion personal y pueden compartirse libremente. Para el desarrollo de algoritmos, se utilizan exclusivamente imagenes sinteticas que contienen datos de prueba generados artificialmente. Este enfoque cumple con GDPR y CCPA porque no se recopilan ni se procesan datos personales. El unico escenario donde se necesitan imagenes reales es para validacion final del motor, y en ese caso las imagenes se procesan localmente en el equipo del usuario, sin enviarse a ningun servidor.

5.5 Tamano del Dataset y Roadmap

El tamano minimo viable del dataset es de aproximadamente 1,000 casos distribuidos en al menos 10 categorias de modo de fallo. El objetivo es alcanzar 5,000 casos con 50-80 variables cada uno, y el aspiracional es 50,000+ casos (escala Backblaze). El problema del desequilibrio de clases es severo: los modos de fallo comunes (eliminacion accidental, corrupcion de MFT) tendran muchos mas ejemplos que los modos raros (fallo de SSD con TRIM, corrupcion de APFS cifrado). Para mitigar esto, se utilizaran tecnicas de sobremuestreo (SMOTE), aprendizaje con costos sensibles, y generacion sintetica dirigida para las categorias infrarrepresentadas.

Fase	Periodo	Casos	Fuente	Tipo
1	Meses 1-3	1,000	Sinteticos	100% ground truth, corruption controlada
2	Meses 3-6	5,000	Talleres de reparacion	Metadatos anonimizados de casos reales
3	Meses 6-12	2,000	Agente de recoleccion	SMART + diagnostico + resultado
4	Meses 6-18	5,000	Universidades	Casos de investigacion forense
5	Meses 12-24	10,000+	Comunidad open-source	Publicar primer dataset publico

Tabla 5.1: Roadmap de construccion del dataset

Capitulo 6: MVP Tecnico — Demostrar que Funciona

El MVP tecnico no es un producto comercial. Es una demostracion de que el motor de recuperacion puede recuperar archivos mejor que los competidores en escenarios especificos y bien definidos. El MVP se limita a NTFS en HDD SATA, y demuestra tres capacidades clave: diagnostico automatizado, imagen priorizada (MFT primero), y checkpoints incrementales. Si el MVP demuestra que estas tres capacidades mejoran objetivamente la tasa de recuperacion, la inversion en desarrollo posterior esta justificada.

6.1 Alcance del MVP

El MVP se define por lo que incluye y, mas importante, por lo que excluye. Las restricciones son intencionales: cada limitacion reduce el tiempo de desarrollo y permite demostrar la propuesta de valor central sin distracciones.

Dimension	Incluido en MVP	Excluido (futuro)
Filesystem	NTFS unicamente	APFS, EXT4, FAT/exFAT, HFS+
Disco	HDD SATA	SSD, NVMe, USB, RAID
Acceso	Solo lectura (imagen)	Escritura, reparacion in-place
Modo	Imagen de disco	Recuperacion directa del disco
Funciones	Diagnostico, imagen priorizada, checkpoints	Carving, reconstruccion, IA de reparacion
Plataforma	Linux (x86_64)	Windows, macOS
Interfaz	CLI (linea de comandos)	GUI
Motor de decisiones	Reglas fijas (no ML)	Red bayesiana, RL

Tabla 6.1: Alcance del MVP tecnico

6.2 Parser NTFS: La Cadena Minima

El parser NTFS del MVP debe seguir una cadena minima de lectura: VBR para obtener la ubicacion del MFT, entrada 0 del MFT para determinar el tamano total de la tabla, iteracion sobre las entradas del MFT para detectar archivos (existentes y eliminados), y lectura de los atributos \$STANDARD_INFORMATION (0x10), \$FILE_NAME (0x30), y \$DATA (0x80) de cada entrada. Para archivos eliminados, el bit de "en uso" en el offset 0x16 esta desactivado, pero los data runs del atributo \$DATA permanecen intactos (si los clusters no fueron reutilizados). El parser tambien debe manejar archivos residentes (datos inline en el MFT) y no residentes (datos en data runs).

Existen crates de Rust que pueden acelerar el desarrollo del parser NTFS: el crate "ntfs" de ColinFinck es maduro y proporciona parsing completo de NTFS, incluyendo soporte para archivos comprimidos, encriptados (EFS), y sparse. El crate "ntfs-core" ofrece un enfoque forense. El crate "mft-rs" es un parser multiplataforma. Usar uno de estos crates como base puede reducir el tiempo de desarrollo del parser NTFS

de 8 semanas a 3-4 semanas, pero requiere evaluar la calidad del codigo y la compatibilidad de licencias.

6.3 Imagen de Disco: Algoritmo de 5 Fases

El motor de imagen del MVP reimplementa el algoritmo de 5 fases de ddrescue con la extension de priorizacion MFT-first. Las 5 fases son: Copying (lectura secuencial rapida de sectores buenos), Trimming (refinamiento de los limites de zonas danadas), Sweeping (escaneo de zonas danadas con reintentos), Scraping (lectura sector a sector en zonas fallidas), y Retrying (reintentos finales con parametros agresivos). La extension MFT-first modifica la fase de Copying: antes de leer secuencialmente, identifica los sectores que contienen el MFT y los lee primero. Esto garantiza que, si el disco falla durante la imagen, las estructuras de metadatos mas importantes ya fueron copiadas.

El estado de cada sector se registra en un bitmap con 3 estados: no procesado, procesado con exito, y procesado con error. El bitmap se codifica con RLE (Run-Length Encoding) para eficiencia: un disco de 18TB tiene aproximadamente 35 mil millones de sectores de 512 bytes, pero las zonas de mismo estado tienden a ser contiguas, lo que permite una compresion muy eficiente. El archivo de mapa se guarda en formato SQLite con escritura atomica para garantizar la integridad de los checkpoints.

6.4 Modulo de Diagnostico: 20 Segundos Antes de Tocar el Disco

El modulo de diagnostico del MVP ejecuta tres operaciones en secuencia, sin escribir nada en el disco: lectura de datos SMART (5 atributos clave), escaneo rapido de superficie (1 sector cada 1000, para detectar zonas danadas sin leer todo el disco), y verificacion del filesystem (leer el VBR y las primeras entradas del MFT para determinar si el filesystem esta intacto). El resultado es un diagnostico con semaforo: verde (disco saludable, proceder con imagen normal), amarillo (disco con signos de debilidad, usar imagen priorizada con reintentos reducidos), o rojo (disco en riesgo de fallo inminente, imagen priorizada inmediata con minimos reintentos). El diagnostico se presenta al usuario con una explicacion clara de la situacion y una recomendacion de accion.

6.5 Stack Tecnico Recomendado

La recomendacion tecnica es Rust para el motor central y Python para el prototipado rapido. Rust ofrece seguridad de memoria (critica cuando se parsean estructuras binarias no confiables de discos danados), rendimiento comparable a C, y un ecosistema de crates maduro para parsing binario (nom, binrw), acceso a disco raw (nix), y serializacion (serde). Python es ideal para prototipar el motor de decisiones y el pipeline de entrenamiento antes de implementarlos en Rust. El desarrollo paralelo con ambos lenguajes permite iterar rapidamente en el diseno del motor de decisiones mientras se construye el motor de recuperacion en Rust.

Componente	Lenguaje	Bibliotecas	Tiempo Estimado
Parser NTFS	Rust	crate ntfs / binrw / nom	4-6 semanas
Lectura raw de disco	Rust	nix (ioctl, O_DIRECT)	3-4 semanas
Motor de imagen	Rust	Reimplementacion ddrescue	6-8 semanas

Componente	Lenguaje	Bibliotecas	Tiempo Estimado
Modulo de diagnostico	Rust	smartctl wrapper / nix	3-4 semanas
Checkpoints	Rust	rusqlite (SQLite)	2-3 semanas
Motor de decisiones (proto)	Python	scikit-learn / pgmpy	4-6 semanas
CLI	Rust	clap	1-2 semanas
Testing y benchmarking	Python + Rust	pytest + cargo test	2-3 semanas

Tabla 6.2: Stack tecnico y estimacion de desarrollo por componente

6.6 Benchmarking: Como Demostrar Superioridad

La demostracion de superioridad requiere un conjunto de pruebas bien definido y metricas objetivas. Se proponen 10 escenarios de prueba que cubren los casos mas comunes de recuperacion, desde un disco saludable con archivos eliminados hasta un disco con MFT severamente danado. Las metricas clave son: tasa de recuperacion (porcentaje de archivos recuperados sobre el total), precision (porcentaje de archivos recuperados con checksum correcto), velocidad (tiempo para primer archivo recuperado, tiempo para completar), seguridad (numero de errores de I/O generados durante la recuperacion), y recuperacion parcial (capacidad de recuperar archivos individuales cuando el disco falla durante el proceso).

Escenario	Descripcion	Metrica Principal
1. NTFS saludable + eliminacion	Archivos eliminados en disco sin dano	Tasa de recuperacion
2. NTFS + sectores danados	Zonas de sectores no legibles	Precision (checksum)
3. NTFS + MFT parcialmente danado	30-40% de entradas MFT danadas	Tasa de recuperacion
4. NTFS + MFT severamente danado	Mas del 70% de MFT danado	Tasa de recuperacion parcial
5. NTFS + disco fallando	Tasa de errores creciente	Tiempo a primer archivo
6. NTFS + disco grande (4TB+)	Imagen con checkpoints	Tiempo de reanudacion
7. NTFS + VBR danado	Sector 0 no legible	Capacidad de deteccion alternativa
8. NTFS + particion perdida	Tabla de particiones danada	Deteccion de particion
9. NTFS + archivo fragmentado	Archivo en 3+ fragmentos no contiguos	Precision de reconstruccion
10. NTFS + disco en riesgo	SMART indica fallo inminente	Priorizacion MFT-first

Tabla 6.3: Escenarios de benchmarking del MVP

6.7 Cronograma de Desarrollo

El desarrollo secuencial del MVP tomara aproximadamente 30 semanas (7.5 meses). Con trabajo paralelo (un desarrollador en Rust y otro en Python), el timeline se reduce a 24 semanas (6 meses). Los componentes mas largos son el parser NTFS (8 semanas si se construye desde cero, 4-6 semanas con un crate Rust existente) y el

motor de imagen (8 semanas incluyendo la reimplementacion del algoritmo de 5 fases y la priorizacion MFT-first). El modulo de diagnostico y los checkpoints pueden desarrollarse en paralelo con el parser NTFS, ya que no dependen de el.

Componente	Semanas	Dependencia	Paralelizable
Parser NTFS	4-6	Ninguna	Si (con diagnostico)
Lectura raw de disco	3-4	Ninguna	Si (con parser)
Motor de imagen	6-8	Lectura raw	Parcialmente
Modulo de diagnostico	3-4	Lectura raw + SMART	Si
Checkpoints	2-3	Motor de imagen	No
CLI	1-2	Todos los anteriores	No
Testing + benchmarking	2-3	Todos los anteriores	No
TOTAL (secuencial)	24-30		
TOTAL (paralelo)	18-24		

Tabla 6.4: Cronograma de desarrollo del MVP

Capitulo 7: Plan de Ingenieria — De la Vision alCodigo

Este capitulo sintetiza los hallazgos de los capitulos anteriores en un plan de ingenieria concreto. El plan identifica exactamente 14 modulos necesarios para el motor de recuperacion completo, de los cuales 6 pueden basarse en componentes existentes y 8 deben desarrollarse desde cero. El modulo mas critico es el motor de decisiones, que no tiene equivalente en ningun producto existente y constituye el principal diferenciador competitivo.

7.1 Mapa de 14 Modulos

El motor de recuperacion requiere 14 modulos para su version completa. El MVP solo implementa los modulos 1-7, que corresponden a la funcionalidad basica de NTFS. Los modulos 8-14 se agregan progresivamente en versiones posteriores. Los modulos marcados como "Reutilizar" pueden basarse en componentes open-source existentes con licencias comerciales. Los modulos marcados como "Desarrollar" deben construirse desde cero y constituyen el valor diferencial del motor.

#	Modulo	Estrategia	Prioridad MVP
1	Raw I/O (lectura cruda)	Desarrollar (Rust + nix)	Si
2	Parser NTFS	Reutilizar (libtsk o crate Rust) + extensiones	Si

#	Modulo	Estrategia	Prioridad MVP
3	Motor de Imagen (5 fases)	Desarrollar (reimplementacion ddrescue)	Si
4	Imagen Priorizada (MFT-first)	Desarrollar (extension del modulo 3)	Si
5	Modulo de Diagnostico	Desarrollar (SMART + superficie + FS)	Si
6	Checkpoints Incrementales	Desarrollar (SQLite + RLE bitmap)	Si
7	CLI	Desarrollar (clap)	Si
8	Parser APFS	Desarrollar (no existe en libtsk)	No
9	Parser EXT4 + journal replay	Desarrollar (extension de libtsk)	No
10	Motor de Carving	Reutilizar (Scalpel) + extensiones	No
11	Motor de Reconstruccion	Desarrollar (heuristica de fragmentos)	No
12	Motor de Decisiones (3 capas)	Desarrollar (Bayes + DT + RL)	No
13	Reparacion de Multimedia	Desarrollar (untrunc + IA)	No
14	GUI / API	Desarrollar (Tauri o Electron)	No

Tabla 7.1: Mapa completo de 14 modulos del motor de recuperacion

7.2 Fases de Desarrollo

El desarrollo se organiza en 4 fases, cada una con entregables claros y criterios de aceptacion. La Fase A (MVP) demuestra que el motor puede recuperar archivos de NTFS mejor que los competidores. La Fase B amplía el soporte a otros filesystems y agrega carving. La Fase C introduce el motor de decisiones, que es el diferenciador principal. La Fase D agrega capacidades avanzadas como reparacion multimedia y GUI.

Fase	Modulos	Duracion	Criterio de Aceptacion
A: MVP	1-7	6 meses	Recuperar NTFS mejor que DiskDrill en 10 escenarios
B: Ampliacion	8-11	4 meses	Soporte APFS + EXT4 + carving funcional
C: Inteligencia	12	4 meses	Motor de decisiones mejora tasa de recuperacion en 15%+
D: Producto	13-14	3 meses	GUI + reparacion multimedia + API

Tabla 7.2: Fases de desarrollo del motor de recuperacion

7.3 Algoritmos Clave: El Corazon del Motor

El motor de recuperacion se basa en tres algoritmos que constituyen su corazon tecnico. El primero es el algoritmo de imagen de 5 fases con priorizacion MFT-first, que extiende el algoritmo de ddrescue con la capacidad de leer primero las estructuras de metadatos del filesystem. El segundo es el motor de reconstruccion de filesystems, que utiliza multiples fuentes de informacion (MFT, INDX, journal, \$Bitmap)

para reconstruir el arbol de archivos cuando el MFT esta parcialmente danado. El tercero es el motor de decisiones de 3 capas (Bayes + arbol de decision + RL), que automatiza las decisiones que hoy toma un tecnico experimentado.

De estos tres algoritmos, el motor de decisiones es el mas innovador y el mas dificil de implementar. La red bayesiana de la Capa 1 requiere un modelo de diagnostico que maneje la incertidumbre de forma robusta, con priors que proporcionen comportamiento razonable desde el primer caso. El arbol de decision de la Capa 2 necesita suficientes datos etiquetados para aprender reglas significativas, lo que se resuelve con el dataset sintetico de la Fase 1 del roadmap. Y la politica de RL de la Capa 3 requiere un simulador de disco que permita entrenar al agente sin acceso a hardware real, lo que se resuelve con el pipeline de generacion de imagenes sinteticas. Cada uno de estos desafios tiene una solucion viable, pero la integracion de las tres capas en un sistema coherente es un desafio de ingenieria significativo.

7.4 Foso Competitivo: Porque No Pueden Copiarnos

El foso competitivo del motor no es un unico algoritmo, sino la acumulacion de datos de recuperacion que mejora el motor de decisiones con cada caso procesado. Un competidor puede copiar la interfaz, puede copiar los checkpoints, puede copiar el diagnostico, pero no puede copiar facilmente: miles de discos analizados con resultados conocidos, millones de errores registrados con contexto, patrones SMART correlacionados con resultados de recuperacion, estrategias que funcionaron y las que no funcionaron, y la retroalimentacion de usuarios confirmando que archivos recuperados son correctos. Este dataset crece con cada usuario del producto, y cada nuevo caso hace que el motor sea marginalmente mejor. Es el mismo efecto de red que hace que Google Maps sea mejor que cualquier competidor: no es que el algoritmo sea necesariamente superior, sino que tienen mas datos de trafico.

El segundo foso es la arquitectura modular. El motor de recuperacion esta disenado como una plataforma, no como un producto. Cada modulo es independiente y puede ser mejorado sin afectar a los demas. El motor de decisiones puede ser reemplazado por un algoritmo mejor sin cambiar los parsers de filesystem. Los parsers pueden ser actualizados sin cambiar el motor de imagen. Esta modularidad permite que el motor evolucione rapidamente y se adapte a nuevos filesystems y nuevos tipos de dano sin reescrituras masivas. Y la arquitectura de plataforma permite que el motor sea utilizado como base para multiples productos: aplicacion de escritorio, version para laboratorios, SDK para fabricantes, y API para servicios de recuperacion en la nube.

7.5 Evaluacion de Riesgos

Los riesgos principales del proyecto son tecnicos, no de mercado. El riesgo tecnico mas significativo es que el motor de decisiones no mejore significativamente la tasa de recuperacion respecto a un sistema de reglas simples. Si despues de entrenar con 1,000 casos, el motor de decisiones no supera a un arbol de decision escrito a mano por un experto, entonces la inversion en ML no esta justificada y el enfoque debe ser reglas codificadas. El segundo riesgo es que el dataset sintetico no capture la complejidad del mundo real, lo que llevaria al motor a tomar decisiones suboptimas en casos reales. El tercer riesgo es que la reimplementacion del algoritmo de ddrescue no alcance la calidad del original, lo que afectaria la capacidad de imagen de discos fallando.

Riesgo	Probabilidad	Impacto	Mitigacion
Motor de decisiones no mejora sobre Media Alto	Alto	Alto	Evaluar con 100 casos sinteticos antes de MVP completo
Dataset sintetico no representa realidad Alta Alta	Alta	Medio	Complementar con datos reales de talleres ASAP
Reimplementacion ddrescue inferior al Alta Alta original	Alta	Alto	Usar ddrescue como subprocesso en MVP, reimplementar despues
Parser NTFS no maneja casos extremos Media Media	Media	Medio	Usar libtsk como base, agregar extensiones propias
No se consiguen datos de talleres	Media	Medio	Plan B: generar mas datos sinteticos + universidades
RL optimiza metrica incorrecta	Media	Medio	Validacion con ground truth + metricas secundarias

Tabla 7.3: Evaluacion de riesgos del proyecto

7.6 Conclusion: La Pregunta Central

Despues de este analisis de ingenieria, la pregunta central que la Fase 3 debe responder es: si podemos construir un motor de recuperacion que sea objetivamente mejor que los existentes, y exactamente por que lo seria. La respuesta preliminar es si, y la razon es triple. Primero, la imagen priorizada (MFT-first) puede mejorar significativamente la tasa de recuperacion en discos fallando, porque asegura que las estructuras de metadatos se copien antes que los datos de usuario. Segundo, el diagnostico automatizado reduce el riesgo de que un usuario sin experiencia empeore la situacion de un disco danado ejecutando un escaneo completo cuando deberia estar creando una imagen. Tercero, el motor de decisiones puede mejorar con el tiempo, acumulando conocimiento que ningun competidor tiene.

Sin embargo, estas ventajas son hipoteticas hasta que se demuestren con un MVP funcional. El MVP no necesita demostrar que el motor de decisiones funciona; solo necesita demostrar que la imagen priorizada y el diagnostico automatizado mejoran los resultados en escenarios especificos. Si el MVP demuestra esto, la inversion en el motor de decisiones esta justificada. Si no, el proyecto puede pivotear hacia un enfoque de reglas codificadas que aun puede ser superior a los competidores por la calidad de la imagen priorizada y los checkpoints incrementales. El proximo paso es construir el MVP y ejecutar los 10 escenarios de benchmarking contra los competidores existentes. Solo los resultados objetivos pueden responder la pregunta central.